

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Анализ безопасности кода.
Часть 2. Методика оценки безопасности кода»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: на основе практических результатов лабораторной работы №3.1 сформулировать многократно применимую методику оценки безопасности кода и проверить её работоспособность, сравнив по ней собственную реализацию с альтернативной, написанной иначе.

Задание:

- а) На основе полученных результатов предыдущей части лабораторной работы создать методику оценки безопасности кода.
- б) По созданной методике сравнить предоставленный алгоритм со своим из предыдущей части лабораторной работы.
- в) Оформить отчёт о проделанной работе.

Постановка задачи. Роль «предоставленного алгоритма» (пункт 4 инструментальных средств задания) играет отдельная, самостоятельно написанная для этой работы программа `Lab3.2-ProvidedAlgorithm` – она реализует ровно ту же функциональность, что и раздел «In-memory data» из лабораторной работы №3.1 (создание/обновление/удаление конфиденциальных и неконфиденциальных данных в оперативной памяти без сохранения на диск), но написана так, как эту задачу решил бы разработчик, не знакомый с выводами лабораторной работы №3.1: без валидации входных данных средствами фреймворка, с обратимым «шифрованием» на основе XOR с зашитым в код ключом вместо AES-256-GCM, с постоянно растущим журналом отладки, в который по ошибке попадают конфиденциальные значения в открытом виде, и без синхронизации доступа к общим структурам данных. Такая программа воспроизводит реальные, часто встречающиеся ошибки, а не гипотетические, и потому пригодна как контрольный пример для проверки создаваемой методики: методика имеет смысл только тогда, когда она способна отличить заведомо небезопасный код от кода, для которого эти вопросы уже были осознанно проработаны.

Описание алгоритмов

Свой алгоритм (лабораторная работа №3.1)

Раздел «In-memory data» приложения `SecureApp` (Lab2.1-SecureApp, добавлен в лабораторной работе №3.1): `InMemoryRecordStore` на `ConcurrentDictionary` с выдачей идентификаторов через `Interlocked.Increment`, конфиденциальные поля шифруются AES-256-GCM (`ConfidentialCrypto`) непосредственно при записи, вся валидация входных данных выполняется декларативно средствами ASP.NET Core (`DataAnnotations` на DTO, автоматическая обработка ошибок разбора

JSON). Подробное описание и обоснование см. в отчёте по лабораторной работе №3.1.

Предоставленный алгоритм

Lab3.2-ProvidedAlgorithm – отдельная консольная программа на том же языке (C#), решающая ту же задачу без использования ASP.NET Core: сырой `System.Net.HttpListener`, обычный (не потокобезопасный) `Dictionary`, ручной разбор JSON без описания схемы и без валидации, «шифрование» через XOR с ключом, зашитым в исходный код (листинг 1), и постоянно растущий список `activityLog`, в который записывается открытый текст каждой операции (листинг 2) – предположительно для отладки, но без учёта того, что содержимое этого списка тоже является конфиденциальными данными.

```
1 string XorObfuscate(string text, bool alreadyBase64 = false)
2 {
3     var key = "my-static-secret-key-123".ToArray();
4     var bytes = alreadyBase64 ? Convert.FromBase64String(text) :
        ↳ Encoding.UTF8.GetBytes(text);
5     for (var i = 0; i < bytes.Length; i++) bytes[i] ^= key[i % key.Length];
6     return alreadyBase64 ? Encoding.UTF8.GetString(bytes) :
        ↳ Convert.ToBase64String(bytes);
7 }
```

Листинг 1 – «Шифрование» через XOR с ключом, зашитым в исходный код

```
1 activityLog.Add($"{DateTime.UtcNow:o} CREATE confidential: title='{title}'
    ↳ content='{content}');
2
3 var id = nextConfId++; // не атомарно
4 confidential[id] = cipher; // обычный Dictionary, без блокировки
```

Листинг 2 – Открытый текст конфиденциального значения, записываемый в постоянно растущий журнал

Единственная обработка ошибок во всей программе – один блок `catch` (`Exception ex`) на верхнем уровне обработчика запроса, возвращающий текст исключения клиенту с кодом 500 независимо от причины сбоя.

Анализ безопасности предоставленного алгоритма

1. Обработка невалидных данных

Тот же набор из семи проверок, что применялся к своему алгоритму в лабораторной работе №3.1, был подан на предоставленный алгоритм (листинг 3). Результат разителен: там, где свой алгоритм для каждой из семи причин возвращает точный, предсказуемый код ответа (400 с описанием конкретной причины или корректный 201), предоставленный алгоритм отвечает 500 с сырым текстом исключения на четыре из семи проверок – вызывающая сторона не может отличить «неверный тип поля» от «пустое тело запроса» иначе как разбором текста исключения, а само это сообщение дополнительно раскрывает внутреннее устройство сервера (пространства имён, сигнатуры методов – классический пример избыточно подробного сообщения об ошибке). Ограничения на размер входных данных нет вовсе: содержимое из 10 050 символов и содержимое размером 5 МБ приняты одинаково успешно (201), что делает сервер уязвимым к исчерпанию памяти при достаточном числе таких запросов.

```
1 == Fault-injection test, own algorithm (Lab 3.1, ASP.NET Core /api/memory/confidential) ==
2 [PASS] wrong JSON types (number/bool instead of string)      -> HTTP 400 (ValidationProblem)
3 [PASS] syntactically malformed JSON body                     -> HTTP 400 (ValidationProblem)
4 [PASS] missing required field ('content' absent)             -> HTTP 400 (ValidationProblem)
5 [PASS] empty request body                                    -> HTTP 400 (ValidationProblem)
6 [PASS] oversized content (10050 chars, limit 10000)         -> HTTP 400 (ValidationProblem)
7 [PASS] very large content (5 MB)                             -> HTTP 400 (ValidationProblem)
8 [PASS] control characters inside content (\n, \t)           -> HTTP 201 (accepted as data)
9 Result: 7/7 differentiated, correct responses. 0 crashes, 0 hangs, 0 stack traces leaked.
10
11 == Fault-injection test, provided algorithm (Lab 3.2, raw HttpListener /confidential) ==
12 [FAIL ] wrong JSON types (number/bool instead of string)    -> HTTP 500 (raw exception message)
13 [FAIL ] syntactically malformed JSON body                   -> HTTP 500 (raw exception message)
14 [FAIL ] missing required field ('content' absent)           -> HTTP 500 (raw exception message)
15 [FAIL ] empty request body                                   -> HTTP 500 (raw exception message)
16 [FAIL ] oversized content (10050 chars)                     -> HTTP 201 (accepted, no limit at all)
17 [FAIL ] very large content (5 MB)                           -> HTTP 201 (accepted, no limit at all)
18 [PASS] control characters inside content (\n, \t)           -> HTTP 201 (accepted as data)
19 Result: 1/7 correct by accident (no crash), 6/7 either leak exception detail (500) or
20 accept unbounded input (201). 0 crashes, 0 hangs, but every failure mode is
21 undifferentiated (identical HTTP 500 for four unrelated causes) or absent (no
22 size limit at all).
```

Листинг 3 – Обработка некорректного ввода: свой алгоритм против предоставленного

2. Потокобезопасность

200 одновременных запросов на создание записи (по 50 в одновременной обработке) были поданы на оба алгоритма (листинг 4). Свой алгоритм обработал все 200 без потерь: 200 уникальных идентификаторов, 200 сохранённых записей. Предоставленный алгоритм вернул 200 ответов, но лишь 199 уникальных идентификаторов – два разных запроса получили один

и тот же идентификатор из-за неатомарного `nextConfId++`, и лишь 198 записей реально сохранились: одна запись была молча перезаписана другой с тем же идентификатором. Это не сбой с сообщением об ошибке, а тихая потеря данных, обнаруживаемая только сравнением числа отправленных и реально сохранённых записей.

```
1 == Concurrency test: 200 concurrent POST requests, 50 in flight at a time ==
2
3 Own algorithm (Lab 3.1, ConcurrentDictionary + Interlocked.Increment):
4   responses received:      200 / 200
5   unique ids returned:     200 / 200
6   records stored afterward: 200 / 200
7   duplicate ids:          none
8   => no data loss, no corruption.
9
10 Provided algorithm (Lab 3.2, plain Dictionary + id++, no lock):
11   responses received:      200 / 200
12   unique ids returned:     199 / 200   (one id issued to two different requests)
13   records stored afterward: 198 / 200   (one of the two colliding writes overwrote the
14   ↪ other)
15   duplicate ids:          id=11 returned to 2 different requests
16   => silent data loss under concurrent writes: 2 of 200 client-visible records
17   never end up durably stored, with no error returned to either client.
```

Листинг 4 – Состояние гонки при 200 параллельных запросах

3. Криптографическая стойкость «шифрования»

XOR с фиксированным, зашитым в исходный код ключом обратим тривиально: доказательство – независимая реализация той же операции на Python (листинг 5), использующая только ключ, скопированный из исходного текста программы, без обращения к запущенному серверу или его памяти. Такое «шифрование» не даёт защиты, если исходный код или скомпилированный файл когда-либо станут доступны стороннему лицу – ровно тот сценарий, для защиты от которого шифрование конфиденциальных данных вообще применяется. Дополнительно, в отличие от AES-GCM, используемого в своём алгоритме, у этой схемы нет ни одноразового вектора (nonce) – одинаковый открытый текст всегда даёт одинаковый шифротекст, – ни проверки подлинности: подмена байт шифротекста напрямую в структуре данных не будет обнаружена при расшифровке.

```
1 KEY = b"my-static-secret-key-123" # copied verbatim from Program.cs
2
3 def xor(data, key):
4     return bytes(b ^ key[i % len(key)] for i, b in enumerate(data))
```

Листинг 5 – Независимое воспроизведение и обращение XOR-«шифрования» по одному лишь ключу из исходного кода

```

1 plaintext: PROVIDED-DUMP-MARKER-Q9F3K2
2 ciphertext: PStiJT0lMS10aSYoM18oNX8gICsAYAt1XjIf
3 recovered: PROVIDED-DUMP-MARKER-Q9F3K2
4 match: True

```

Листинг 6 – Результат восстановления открытого текста без обращения к серверу

4. Работа с конфиденциальными данными в оперативной памяти

Дампы обоих алгоритмов снимались по методике, аналогичной лабораторной работе №3.1, но с уточнением: поиск утилитой `strings` велся в двух режимах – 8-битном (находит только байтовые, как правило UTF-8, буферы) и `-e 1` (UTF-16LE, необходим, чтобы увидеть сами объекты `System.String` в управляемой куче .NET, которые в 8-битном режиме невидимы). Это уточнение самой методики поиска – не использованное в лабораторной работе №3.1 – позволило впервые различить временные буферы фреймворка от долгоживущих объектов приложения (листинг 7).

```

1 == Memory-dump comparison: confidential marker occurrences, dual-encoding scan ==
2 (strings -e s = 8-bit/UTF-8 scan; strings -e l = UTF-16LE scan, needed to see
3   raw .NET `string` objects, which strings' default 8-bit mode cannot find)
4
5 Own algorithm (Lab 3.1, SecureApp /api/memory/confidential, AES-256-GCM):
6   marker: SECUREAPP-RECHECK-MARKER-W7H2
7           UTF-8   UTF-16LE
8   after create:      3       2
9   after delete + forced GC: 3       2      (unchanged -- see explanation below)
10
11 Residual matches identified as: the raw HTTP request/response JSON bodies
12 (UTF-8 socket buffers) and the deserialized/decrypted .NET string objects
13 used to build that one response (UTF-16LE). None are application-level
14 data structures -- InMemoryRecordStore itself holds only AES-GCM ciphertext.
15 Explanation: ASP.NET Core/Kestrel/System.Text.Json reuse pooled buffers
16 (ArrayPool<byte>/ArrayPool<char>) for request/response processing; a
17 buffer returned to the pool is not garbage from the GC's point of view
18 (the pool still references it) and is not zeroed on return, so it
19 survives even a forced full GC. This is a platform characteristic, not
20 an application-level defect.
21
22 Provided algorithm (Lab 3.2, raw HttpListener /confidential, static-key XOR):
23   marker: PROVIDED-DUMP-MARKER-Q9F3K2
24           UTF-8   UTF-16LE
25   after create:      6       7
26   after delete + forced GC: 2       2      (reduced, but not to zero)
27
28 The 2 residual UTF-16LE matches after delete + forced GC were extracted
29 and read verbatim:
30   "...CREATE confidential: title='dump-test' content='PROVIDED-DUMP-MARKER-Q9F3K2'"
31   "...DELETE confidential#3title='dump-test' content='PROVIDED-DUMP-MARKER-Q9F3K2'"
32 These are literal entries in the program's own `activityLog` list, which
33 is never trimmed or cleared. Unlike the own algorithm's residue, this is
34 an avoidable, application-level defect (logging the confidential value in
35 cleartext) -- not a platform artifact -- and it persists for the entire
36 lifetime of the process, regardless of how many times the record is
37 updated, deleted, or how many full garbage collections run.

```

Листинг 7 – Число вхождений маркера до/после удаления и принудительной сборки мусора

У своего алгоритма число вхождений маркера не изменилось после удаления записи и принудительной полной сборки мусора (3 и 2 – в 8-битном и 16-битном режиме соответственно, для обоих дампов одинаково) – все найденные копии оказались буферами конвейера ASP.NET Core, а не структурами данных самого хранилища. У предоставленного алгоритма число вхождений заметно уменьшилось после сборки мусора (с 6 до 2 и с 7 до 2 соответственно) – то есть большая часть временных копий действительно была собрана, – но два вхождения остались навсегда: это оказались буквальными строки из `activityLog`, что было подтверждено прямым чтением найденных строк из дампа. Иначе говоря, у предоставленного алгоритма после удаления записи и явной сборки мусора конфиденциальное значение остаётся в памяти процесса из-за собственной, легко устранимой ошибки кода (логирование секрета), тогда как у своего алгоритма остаток объясняется исключительно платформой и не устраним на уровне кода приложения без отказа от использующегося фреймворка целиком.

Разработанная методика оценки безопасности кода

Методика построена как контрольный список из шести критериев, каждый – с конкретной, воспроизводимой техникой проверки, выведенной из практического опыта лабораторных работ №2.1–3.1, и трёхуровневой оценкой (соответствует / частично соответствует / не соответствует).

а) **Устойчивость к невалидному вводу.** Подать не менее шести видов заведомо некорректного ввода (неверный тип, синтаксическая ошибка, отсутствующее поле, пустое тело, превышение разумного размера, управляющие символы) и проверить: (а) отклоняется ли каждый некорректный случай явным, специфичным для причины кодом ответа, а не общим сбоем; (б) не раскрывается ли во внешнем ответе внутренняя информация (текст исключения, путь к файлу, версия фреймворка).

б) **Потокобезопасность разделяемого состояния.** Подать не менее 100–200 одновременных операций записи и сравнить число отправленных запросов, число уникальных возвращённых идентификаторов и число реально сохранившихся записей – любое расхождение означает состояние гонки.

в) **Криптографическая стойкость защиты конфиденциальных данных.** Определить алгоритм и источник ключа по коду; проверить наличие одноразового вектора/соли на каждую операцию (то есть разные ли шифротексты получаются из одинакового открытого текста) и наличие проверки подлинности (защиты от подмены шифротекста); попытаться независимо, без обращения к работающему серверу, воспроизвести обратное преобразование по одному лишь исходному коду.

г) **Устойчивость к нагрузке по размеру входных данных.** Определить, существует ли явный верхний предел размера принимаемых данных, и подать вход, заметно превышающий разумный размер (на два–три порядка), проверив код ответа и то, что процесс не потребляет ресурсы неограниченно.

д) **Работа с конфиденциальными данными в оперативной памяти.** Снять дампы процесса после создания, после изменения и после удаления конфиденциальной записи с уникальным маркером; выполнить двухрежимный поиск маркера в каждом дампе (8-битный и `strings -e 1`, то есть UTF-16LE) до и после принудительной полной сборки мусора. Если после сборки мусора маркер остаётся – установить, является ли источник кодом самого приложения (структуры данных, журналы) или инфраструктурой платформы, поскольку от этого зависит, устранима ли находка в принципе на уровне кода приложения.

е) **Изоляция сбоев.** Убедиться, что некорректный запрос не влияет на обработку последующих корректных запросов и не приводит к зависанию или падению процесса в целом.

Обоснование корректности методики: каждый критерий сформулирован как воспроизводимая, автоматизируемая проверка с однозначно интерпретируемым результатом (числовое сравнение или бинарный факт «найдено/не найдено»), а не как экспертная оценка «на глаз» – поэтому применение методики к одному и тому же алгоритму разными людьми в разное время должно давать одинаковый результат. Дополнительно методика была опробована на двух существенно разных по качеству реализациях, продемонстрировав уверенное разделение (см. далее) – то есть не является тривиальной или всегда дающей одинаковый ответ вне зависимости от входа.

Сравнение своего варианта с предоставленным по методике

Критерий	Свой алгоритм (№3.1)	Предоставленный алгоритм
1. Невалидный ввод	Соответствует (7/7 точных кодов, 0 утечек деталей)	Не соответствует (4/7 – общий 500 с текстом исключения)
2. Потокбезопасность	Соответствует (0 потерь из 200)	Не соответствует (2 записи из 200 потеряны молча)
3. Криптостойкость	Соответствует (AES-256-GCM, поппе, тег подлинности)	Не соответствует (обратимый XOR, статический ключ, без поппе и без тега)
4. Пределы размера входа	Соответствует (400 уже на 10 050 симв.)	Не соответствует (принят вход 5 МБ без ограничений)
5. Конфиденциальность в памяти	Частично (остаток есть, но не устраним и не является ошибкой кода)	Не соответствует (остаток – прямое следствие собственной ошибки: логирование секрета)
6. Изоляция сбоя	Соответствует (сервер работоспособен после всех проверок)	Соответствует (сервер работоспособен после всех проверок, хотя и ценой недифференцированных 500)

Таблица 1 – Сравнение по разработанной методике

Из шести критериев предоставленный алгоритм полностью соответствует только одному (изоляция сбоя – сервер не падает целиком, хотя и обрабатывает ошибки крайне грубо), частично или полностью не соответствует остальным пяти. Свой алгоритм полностью соответствует пяти критериям из шести и лишь частично – шестому, причём именно потому, что этот шестой пункт (остаточное присутствие данных в памяти после сборки мусора) для управляемой платформы в принципе неустраним средствами прикладного кода, что и зафиксировано методикой отдельно от устранимых нарушений.

Выводы

В ходе выполнения лабораторной работы на основе практических результатов лабораторной работы №3.1 разработана методика оценки безопасности кода из шести воспроизводимых, автоматизируемых критериев (устойчивость к невалидному вводу, потокобезопасность, криптографическая стойкость, пределы размера входа, работа с конфиденциальными данными в памяти, изоляция сбоев). Методика применена для сравнения собственной реализации из лабораторной работы №3.1 с предоставленным алгоритмом – отдельно написанной программой, воспроизводящей типичные ошибки разработчика, не знакомого с этими критериями. Сравнение показало уверенное и содержательное разделение: собственная реализация полностью соответствует пяти критериям из шести, предоставленный алгоритм – лишь одному.

Семантически работа показала, что методика оценки безопасности кода приобретает практическую ценность только тогда, когда каждый её критерий сформулирован как конкретный, воспроизводимый эксперимент с измеримым результатом, а не как общее пожелание вроде «код должен быть безопасным». Показательно, что один и тот же критерий – остаточное присутствие конфиденциальных данных в памяти после удаления и сборки мусора – дал ненулевой результат для обоих алгоритмов, но методика, включающая уточнённый (двухрежимный, учитывающий кодировку UTF-16LE) поиск и последующую идентификацию источника остатка, позволила корректно различить неустранимую особенность платформы (свой алгоритм) и устранимую ошибку кода (предоставленный алгоритм) – без этого уточнения оба случая выглядели бы одинаково «плохими», хотя по существу требуют совершенно разных действий: для своего алгоритма – ничего (граница возможностей платформы), для предоставленного – удаления логирования секрета из кода.

С прагматической точки зрения работа показала, что сравнение по чёткой методике превращает общее, трудно оспоримое утверждение «моя реализация безопаснее» в конкретный, проверяемый третьей стороной результат: 7 из 7 против 3 из 7 корректных ответов на некорректный ввод, 0 против 2 потерянных записей из 200 при параллельной записи, работающее против тривиально обратимого шифрование – каждое утверждение подкреплено воспроизводимым экспериментом, а не мнением автора о собственном коде. Также показано, что методика, однажды созданная для одной пары реализаций, переносима: ни один из шести критериев не привязан к конкретному языку, фреймворку или предметной области, и та же методика в дальнейшем может быть применена к любому новому алгоритму, работающему с конфиденциальными данными в оперативной памяти.